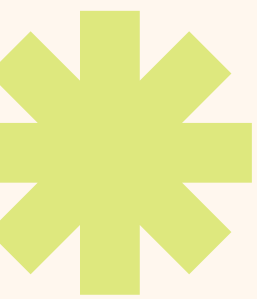


YIELDLENS: STABILITY-AWARE SENSOR REDUCTION FOR SEMICONDUCTOR YIELD MONITORING USING CHRONOLOGICAL DRIFT VALIDATION



Karthick S
Author



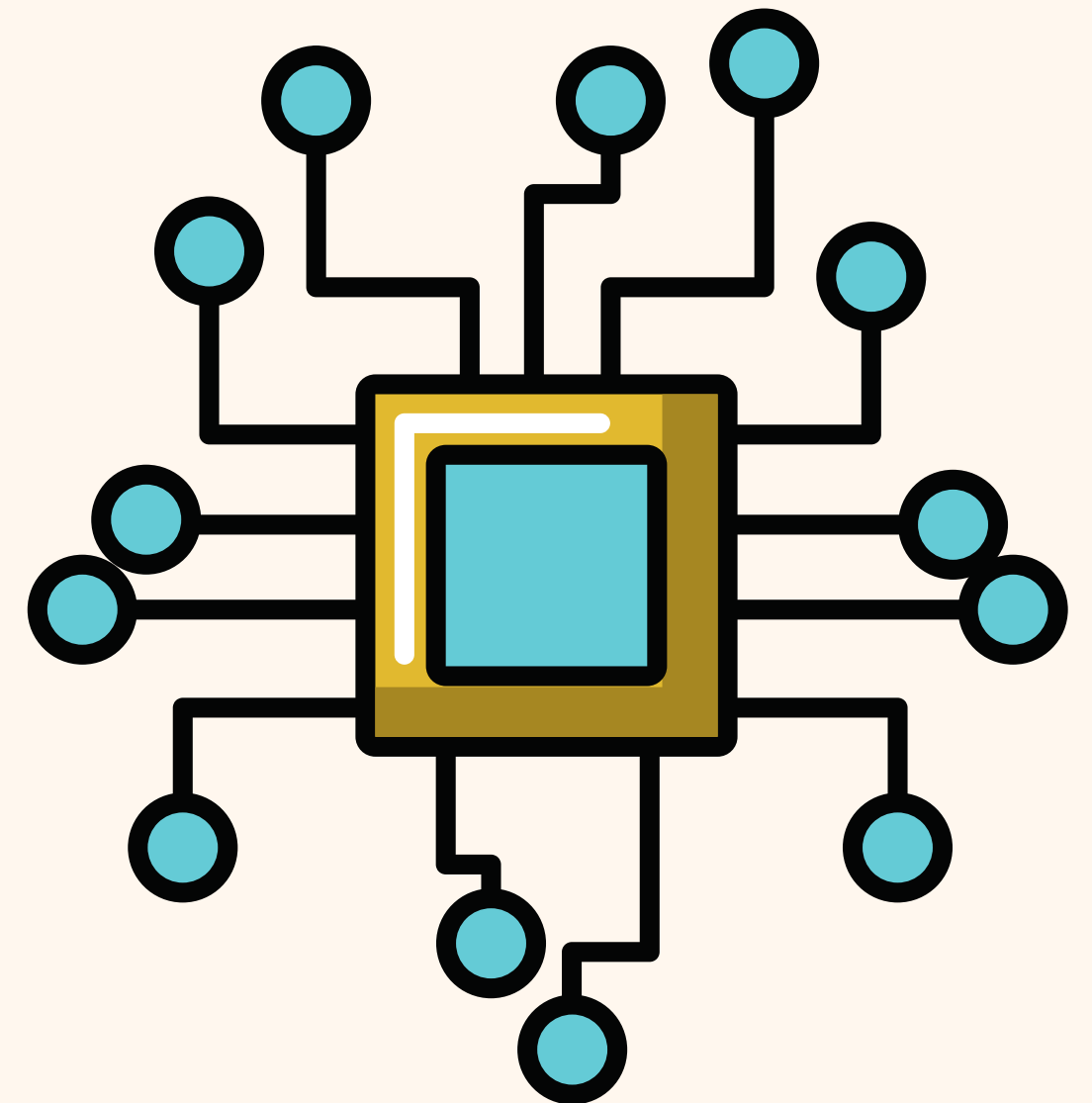
Karthikeyan S
Author



Savithri R
Co - Author



Selvarani K
Co - Author



Problem Statement

Existing feature selection methods rank sensors using a single snapshot of data, assuming that sensor importance remains constant over time. However, in semiconductor manufacturing, sensor behavior changes due to process drift, making these methods unreliable for long-term fault prediction. This challenge is further compounded by the highly imbalanced nature of the data, where failure samples are rare.

Key Challenges

Identifying a small set of sensors that stays predictive over time, not just in one snapshot — while also not discarding rare but critical sensors that only "activate" during failure events.



Research Gap

- Most SECOM studies perform static feature selection using a single train-test split.
- The timestamp information is largely ignored, despite process changes over time.
- Existing stability-selection methods test robustness through random sampling, not real chronological process drift.

Research Objective

To design a sensor reduction method that selects sensors based on their predictive stability across chronologically ordered time batches, rather than importance on a single snapshot — producing a smaller, drift-aware sensor shortlist that is at least as reliable as the full sensor set.

Dataset & Preprocessing

Dataset: SECOM (UCI Machine Learning Repository)

Total records: 1,567 wafers

Features

- **Original: 590 sensor readings** (+ timestamp + pass/fail label)
- **After tripwire audit: 128 near-constant sensors flagged, 2 whitelisted**
- **Final shortlist: 17 sensors**

Outcome classes: 2 (Pass / Fail) — class imbalance ~93:7 (fail rate ~6.6%)



Data Split

- **Chronological Batching:** 5 sequential, non-overlapping batches.
- **Train/Test Split:** Batches 0–3 for training, Batch 4 for testing.
- **Class Imbalance:** SMOTE on training data only; class weighting during LASSO



Preprocessing Pipeline

Missing Value Handling

- Per-sensor median imputation
- Preserved all 590 sensors



Duplicate / Leakage Prevention

- Chronological batching (no random shuffling)
- Prevented future data leakage



Feature Selection & Reduction

- **LASSO per batch → Stability Score**
- **Tripwire Audit rescues rare but important sensors**



Chronological Splitting

- Past → Future evaluation
- Simulates real-world deployment



Feature Scaling

- Applied within each training batch
- Prevented scaling leakage across time



Literature Review & Research Gap



Paper	Approach	Limitation	Our Work (YieldLens)
McCann & Johnston (2008)	Baseline SECOM feature selection	Static single split	Chronological batching on all 590 sensors
Guyon & Elisseeff (2003)	Feature selection survey	No temporal analysis	Drift-aware embedded feature selection
Tibshirani (1996)	LASSO feature selection	No stability over time	LASSO applied across time batches
Meinshausen & Bühlmann (2010)	Stability selection	Random resampling only	Stability across chronological batches
Gama et al. (2014)	Concept drift survey	Not for sensor selection	Applies drift to sensor stability
He & Garcia (2009)	Imbalanced learning	General methods	SMOTE + class-weighted learning
Chambers (2026)	Practical SECOM analysis	Informal methodology	Formal Tripwire Sensor Audit

Models Overview

XGBOOST

What: Gradient-boosted decision tree ensemble for supervised binary classification (pass/fail)

Architecture:

- Trees built sequentially, each correcting errors of the previous
- Class imbalance handled via SMOTE (train set only)

How: Each new tree fits the residual errors of prior trees → final prediction = weighted sum across all trees

Why: Handles high-dimensional, noisy sensor data well; robust to redundant features.

Key Insight — Full Sensor Set (590):

- AUC: 0.605
- Precision@30: 0.133
- Recall@30: 0.235

Key Insight — Reduced Shortlist (17):

- AUC: 0.692 (↑ 0.087)
- Precision@30: 0.167 (↑ 0.034)
- Recall@30: 0.294 (↑ 25%)

RANDOM FOREST

What: Bagged ensemble of decision trees for supervised binary classification (pass/fail).

Architecture:

- Many trees trained independently on bootstrapped samples of the data
- Final prediction = majority vote / averaged output across trees

How: Each tree sees a random subset of data + features → averaging reduces variance and overfitting.

Why: Different training strategy than XGBoost (parallel bagging vs. sequential boosting) — confirms the improvement isn't specific to one algorithm.

Key Insight — Full Sensor Set (590):

- AUC: 0.593
- Precision@30: 0.100
- Recall@30: 0.176

Key Insight — Reduced Shortlist (17):

- AUC: 0.655 (↑ 0.062)
- Precision@30: 0.200 (↑ 100%)
- Recall@30: 0.353 (↑ 100%)

System Architecture

Input

- 590 Sensor Readings + Labels
- Chronologically Sorted Data

Data Processing

- Median Imputation
- Tripwire Sensor Audit
- Chronological Batching (5)

Sensor Selection

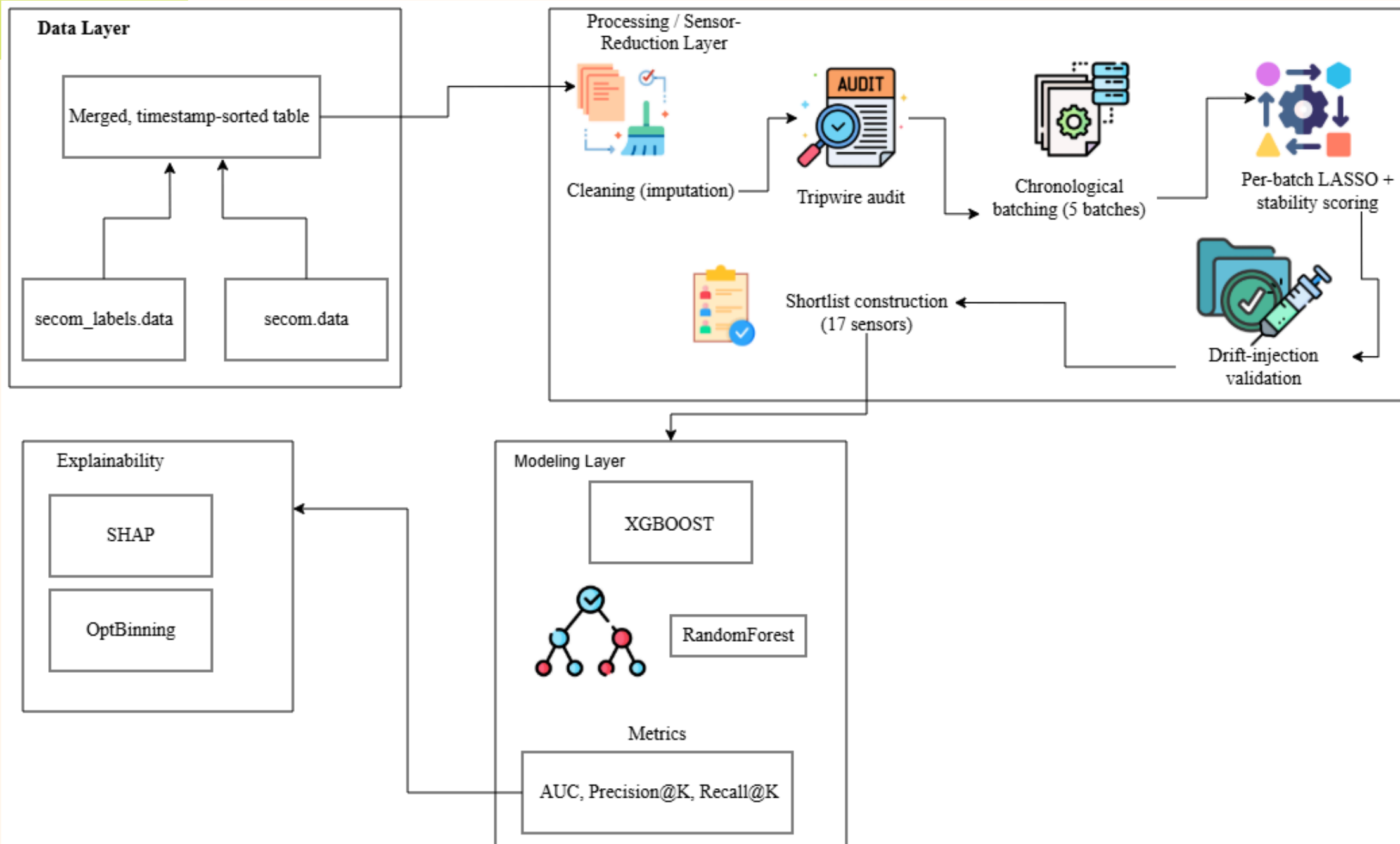
- LASSO Feature Selection
- Stability Score Calculation
- Drift Validation
- 17-Sensor Shortlist

Model Training

- XGBoost & Random Forest
- SMOTE (Training Only)
- Chronological Train/Test Split

Explainability

- SHAP Feature Importance
- Risk Bin Generation



Key Techniques

- **Chronological Batching** – Prevents data leakage.
- **LASSO Stability Selection** – Selects consistent sensors.
- **Tripwire Audit** – Preserves critical rare sensors.
- **SHAP Analysis** – Explains model predictions.



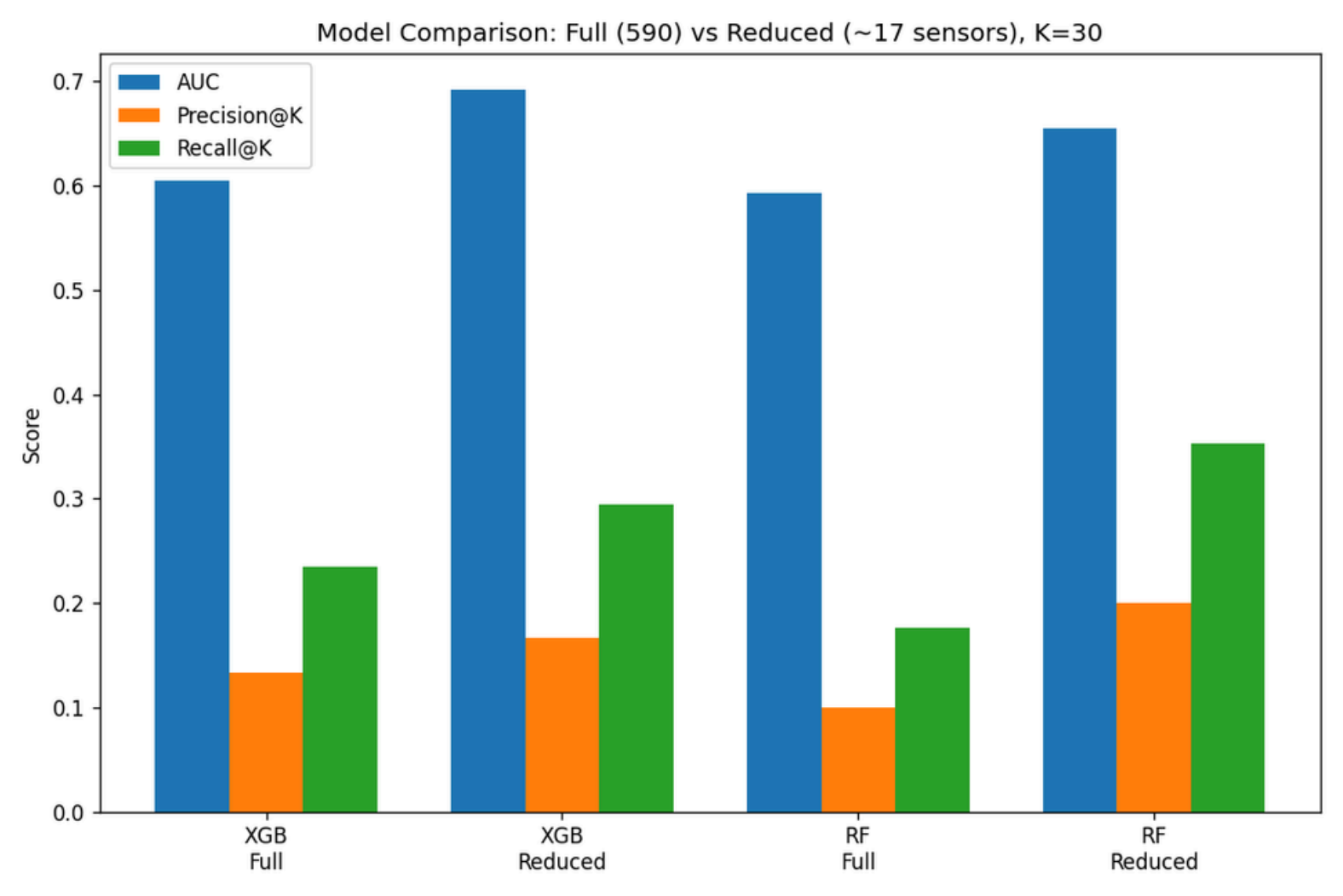
Model Performance

RESULTS

Model	Sensor Set	AUC	Precision@30	Recall@30
XGBoost	590	0.605	0.133	0.235
XGBoost	17	0.692	0.167	0.294
Random Forest	590	0.593	0.1	0.176
Random Forest	17	0.655	0.2	0.353

Highlight

- 17-sensor model outperformed the full 590-sensor model.



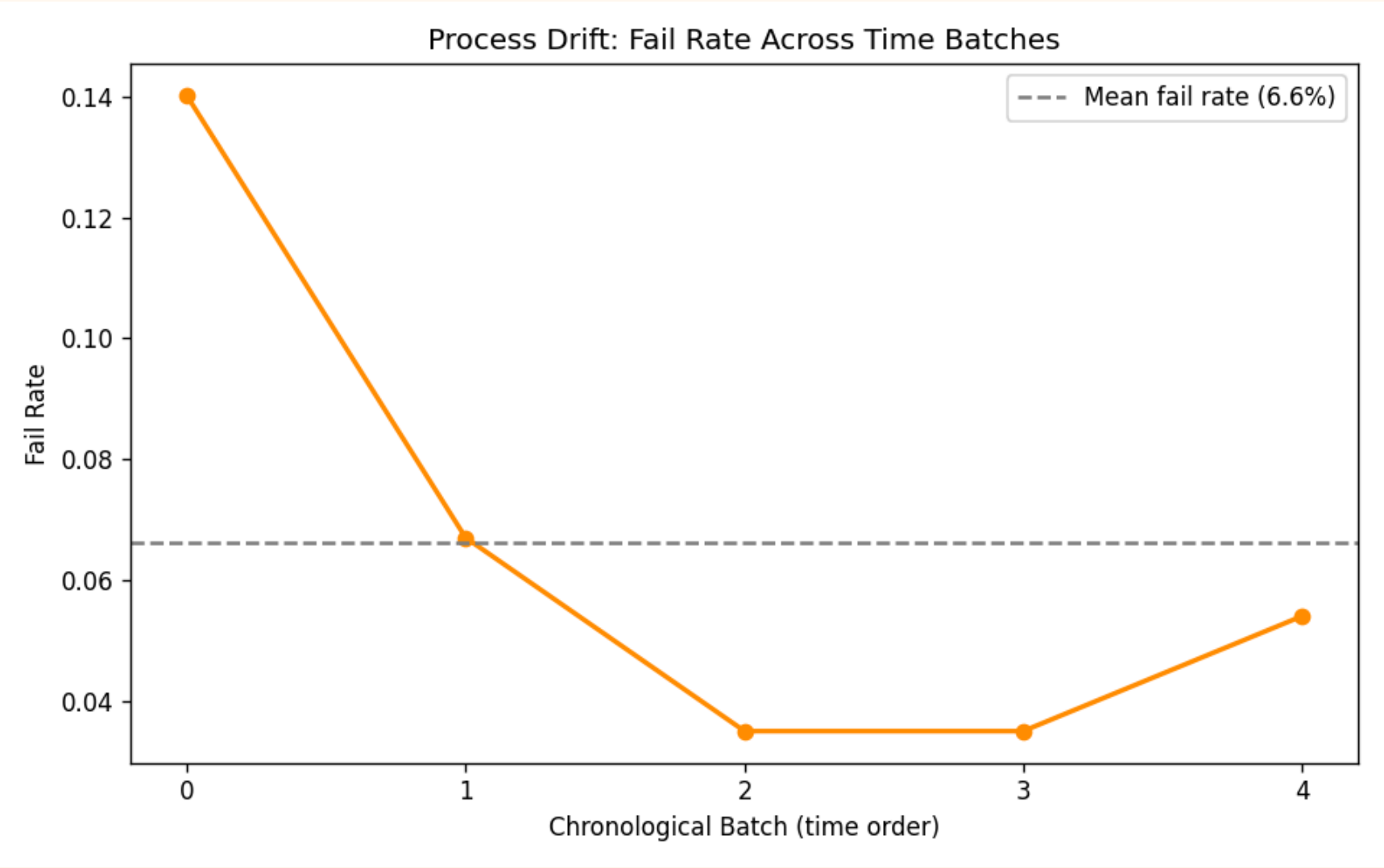
Process Drift

Purpose: Show why a drift-aware approach is needed.

Content

- Failure rate changes across chronological batches, confirming process drift.
- Mention that failure rate drops from 14.0% to 3.5%, then rises to 5.4%.

Batch	Rows	Failures	Fail Rate
0	314	44	14.00%
1	313	21	6.70%
2	313	11	3.50%
3	313	11	3.50%
4	314	17	5.40%

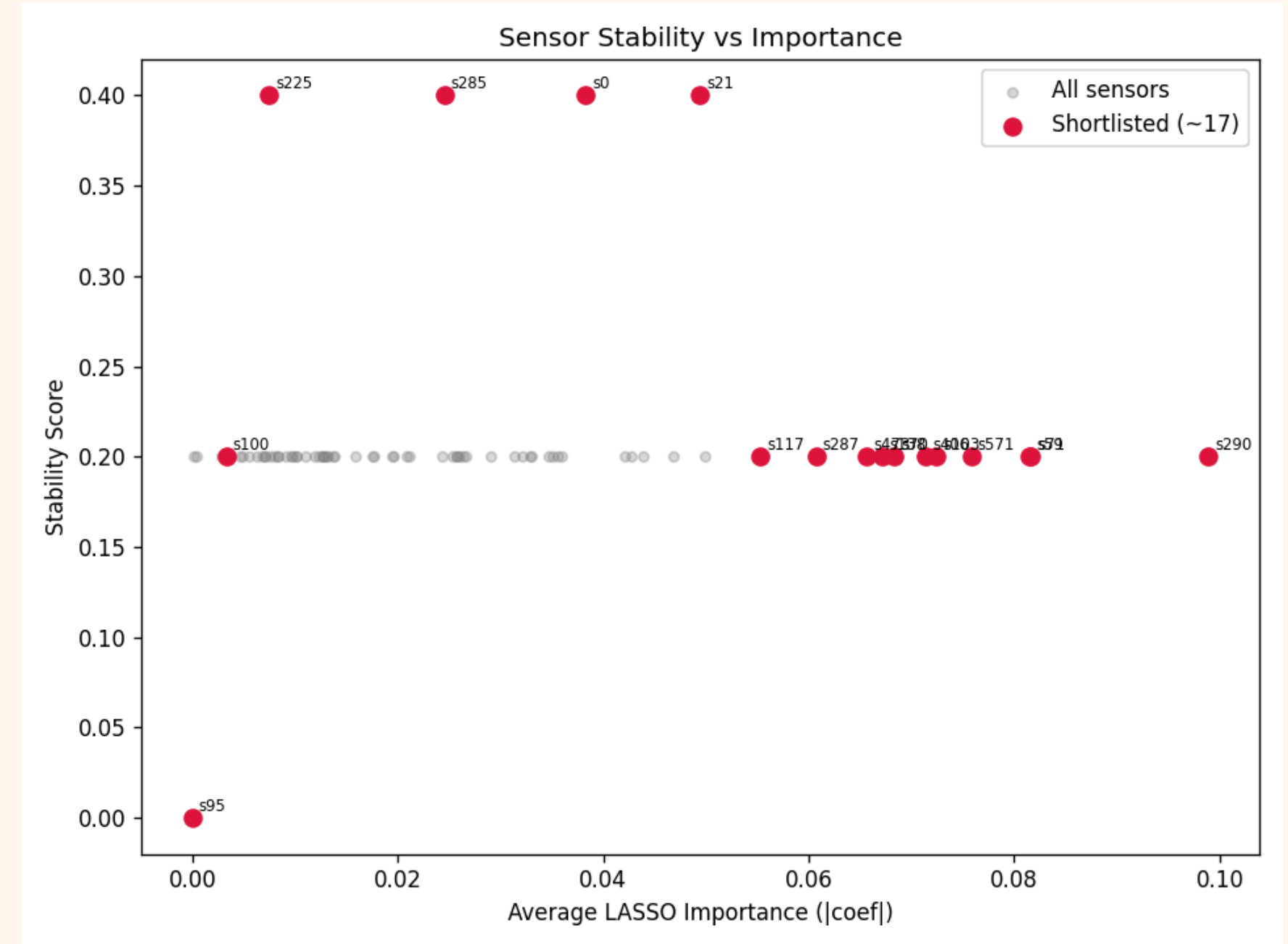


Sensor Stability

Purpose: Show how your method selected sensors.

Content

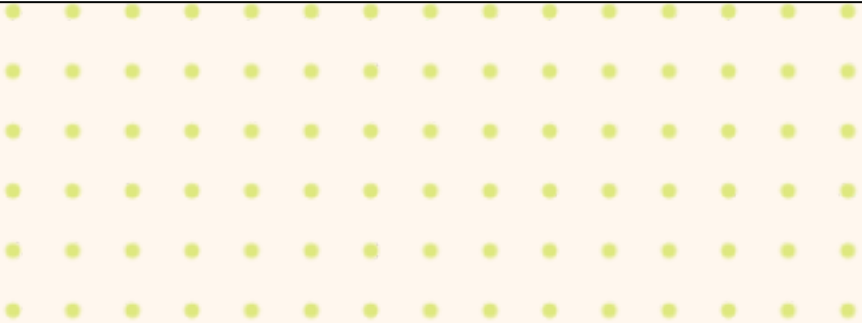
- No sensor exceeded a stability score of 0.4.
- **Top stable sensors:**
 - sensor_0, sensor_21, sensor_225, sensor_285, sensor_95 was retained through the Tripwire Audit despite a stability score of 0.0.



Drift Validation

Purpose: Prove that the stability score actually responds to drift.

Condition	Stability Score	Selected?
Before injection	0.4	Yes
After noise injection	0.2	No



Explainability

Purpose: Show that the model is interpretable.

Rank	Sensor	Mean SHAP
1	sensor_59	0.143
2	sensor_290	0.051
3	sensor_287	0.051
4	sensor_103	0.04
5	sensor_95	0.039

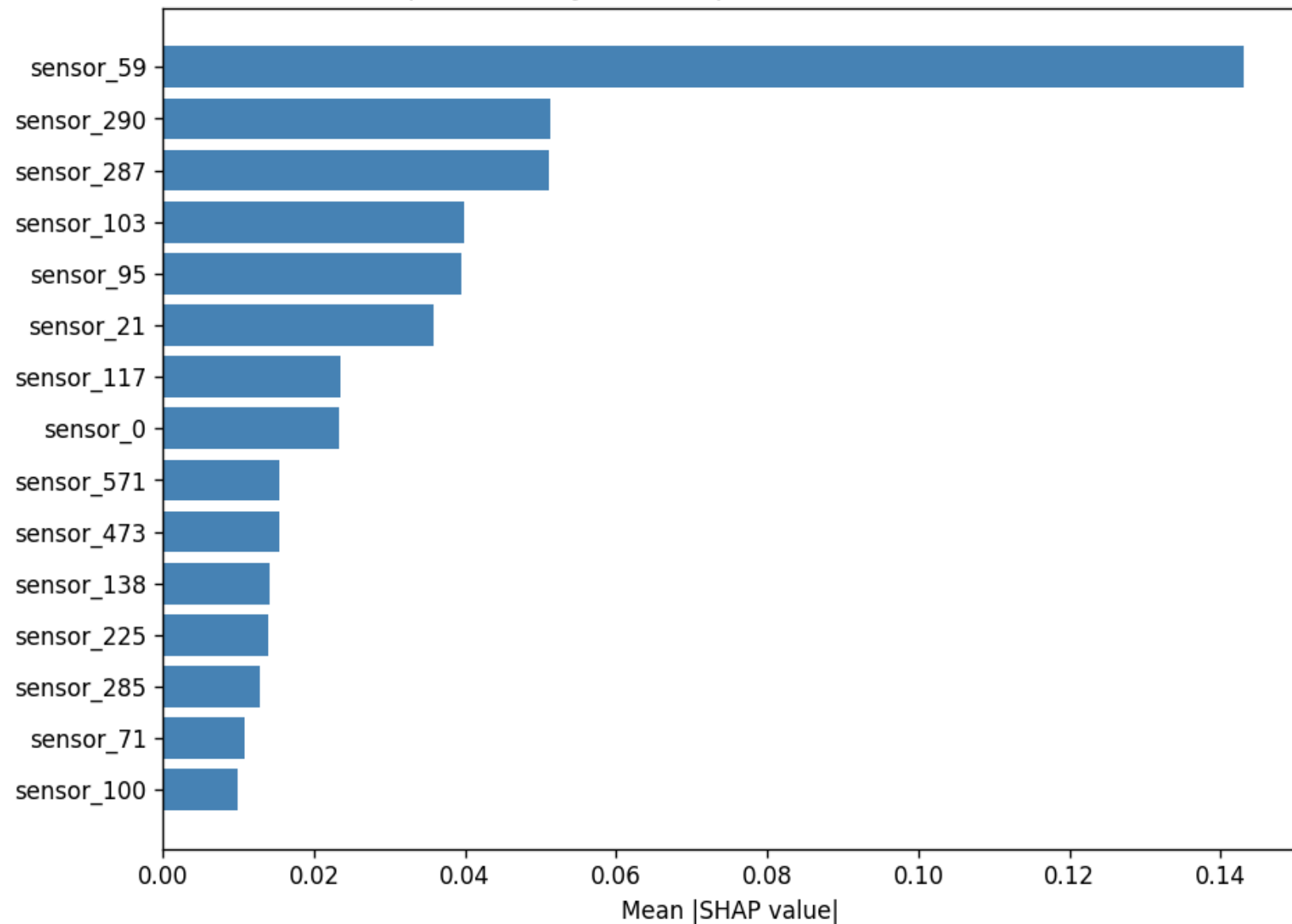
Key Finding

- sensor_59 risk range: 0.9% → 19.0% failure rate



Explainability

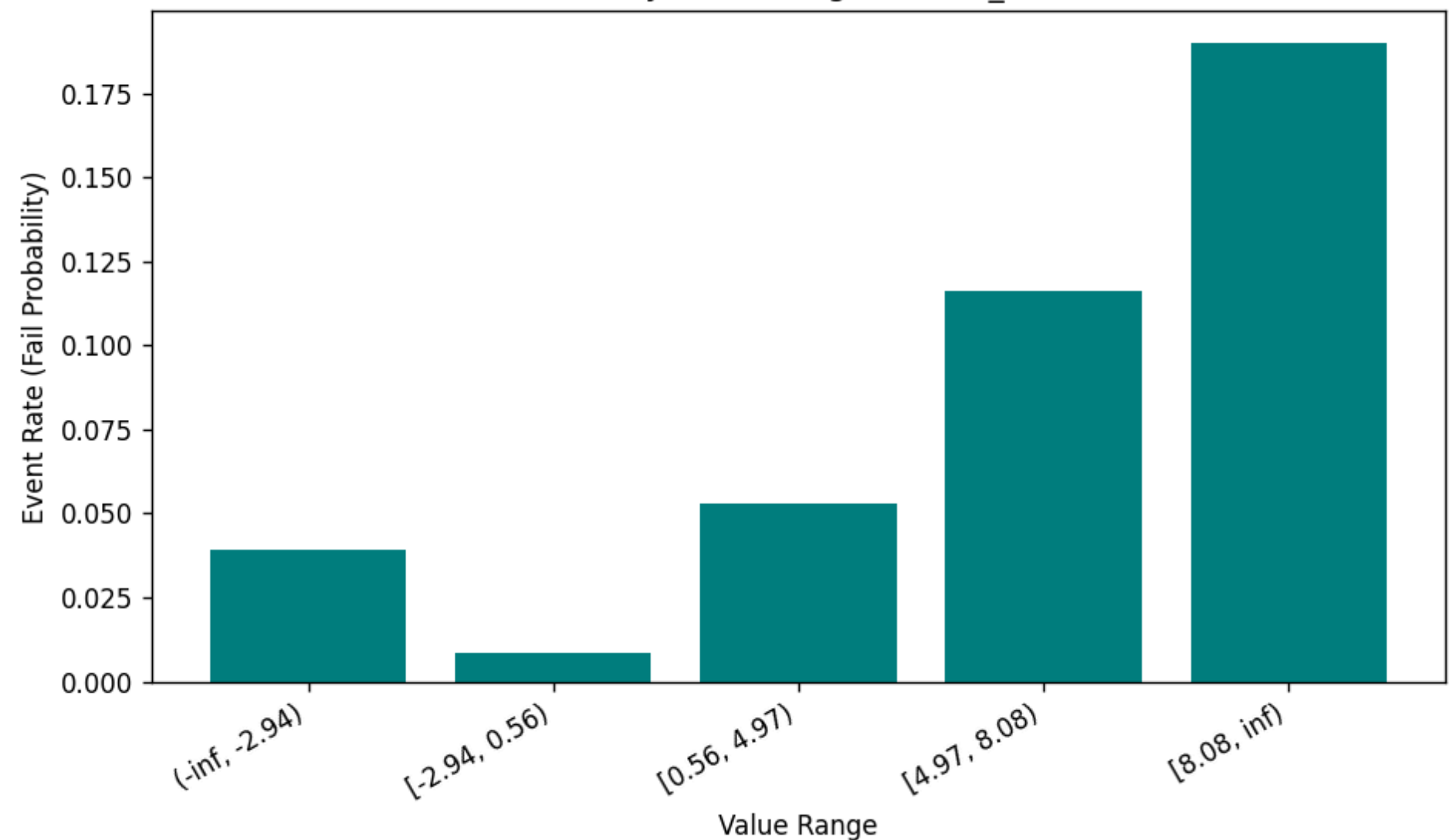
Top Sensors by SHAP Importance (Reduced Model)



Mean absolute SHAP value per sensor, reduced model, ranked highest to lowest.

Observed failure rate by value range for sensor_59, the top-ranked sensor by SHAP importance

Risk by Value Range: sensor_59

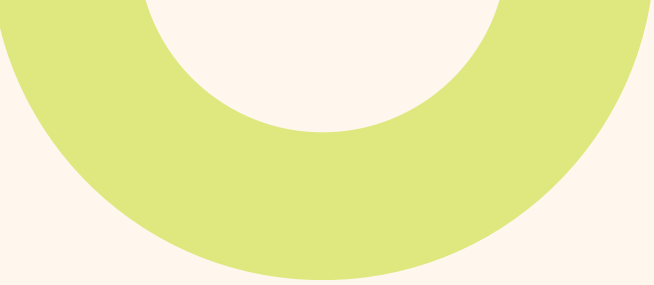


Key Findings

- 17 sensors outperformed all 590 sensors across AUC, Precision@30, and Recall@30.
- Process drift exists, with failure rates changing from 14.0% → 3.5% → 5.4% across time.
- No sensor remained consistently predictive (maximum stability score = 0.4).
- Random Forest achieved the biggest improvement, doubling both Precision@30 and Recall@30.
- Drift injection validated that the stability score responds to changes in sensor behavior (0.4 → 0.2).
- Tripwire Audit preserved critical sensors, retaining sensor_95, which later ranked 5th in SHAP importance.

Core Takeaway

A drift-aware 17-sensor shortlist improved prediction accuracy while reducing the sensor set by ~97%.



Conclusion



YieldLens cuts sensor monitoring from 590 to 17 by selecting sensors based on predictive stability across time, not importance on one snapshot. The reduced set beat the full set across both models and all metrics, and a drift-injection test confirmed the stability score genuinely tracks drift. A separate tripwire audit recovered a key sensor that stability selection alone would have missed — showing one selection method isn't enough on its own.

Future Work

- Test on additional fabrication datasets beyond SECOM
- Add correlation-clustering to catch multi-sensor redundancy
- Evaluate stability across more chronological splits as more data becomes available

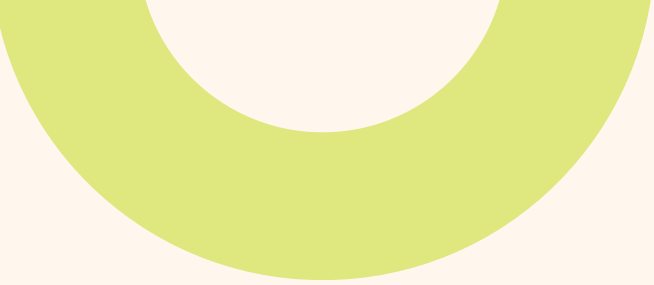


Q&A



Any Questions?





Thank You

